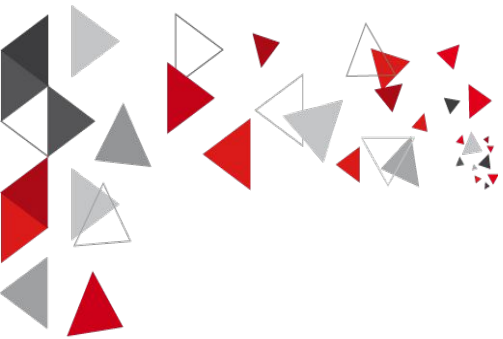
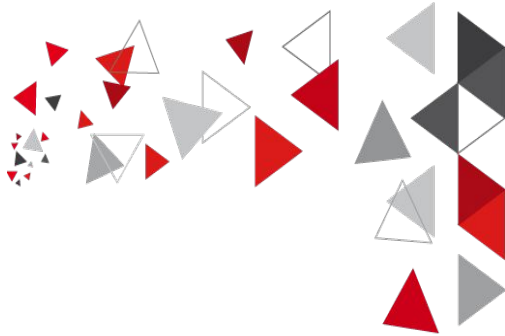


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orienté Objet et Programmation Java

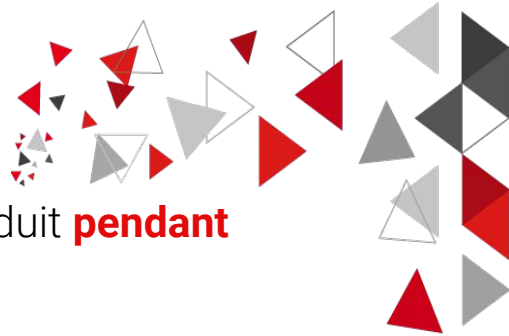
Chapitre 7: Exceptions

Objectifs du chapitre



- Découvrir la notion des exceptions
- Savoir écrire un gestionnaire d’exception avec le bloc try
- Savoir le cheminement d’une exception
- Différencier l’utilisation des deux mots clés throw et throws
- Comprendre l’utilisation du multicatch

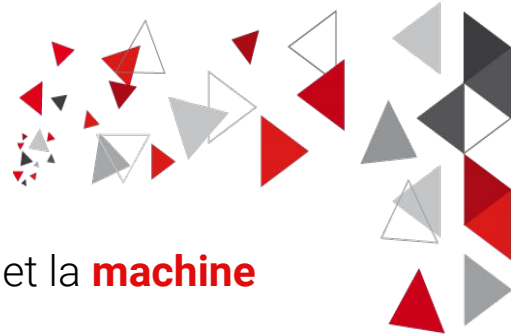
Exception: Définition



Une exception en Java, est un événement ou une condition qui se produit **pendant l'exécution** d'un programme et **perturbe** le flux normal du programme.

Les exceptions sont utilisées pour signaler **une erreur** ou **une situation inattendue** qui peut **empêcher** l'exécution normale du programme.

Exception : Quand?



Une exception peut être déclenchée de deux façons :

- L'exécution du programme ne se déroule pas de la façon prévue, et la **machine Java (JVM)** génère une exception.

- Le programme décide lui-même de déclencher une exception, afin de signaler à la méthode appelante que quelque chose ne se déroule pas comme prévu.

Exception : exemple(1/2)



Bien que ce code ne présente aucune erreur de compilation, une exception sera déclenchée lors de l'exécution car il est interdit de diviser par zéro.

```
public class TestException{

   public static void main(String[] args) {
     int x = 5;
     int y = 0;

     System.out.println("Avant incident");
     System.out.println("La division de x/y est :" + x / y);
     System.out.println("Après incident");

   }
}
```

Exception : exemple(2/2)



Dès que l'instruction "x/y" a été exécutée, elle a provoqué un incident. En effet, une exception de la classe **ArithmeticException** a été "lancée" (un objet de cette classe a été instancié) par la JVM. Cette exception **empêche** le reste du code de s'exécuter.

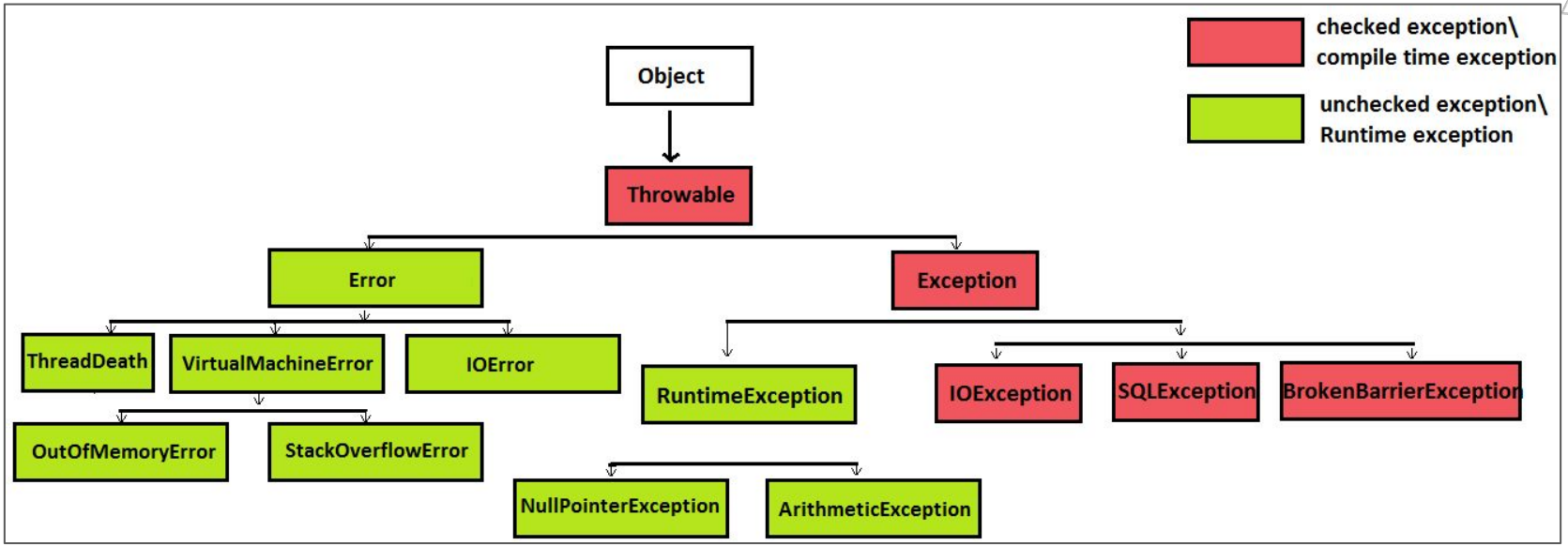
```
Output:
Avant incident
Exception in thread "main" java.lang.ArithmeticException:
/ by zero at TestException.main(TestException.java:7)
```

erreur de compilation & erreur d'exécution



L'erreur de compilation est une erreur qui empêche la compilation du programme, par contre une erreur d'exécution arrive au "Runtime" ou à l'execution, et il se manifeste par la levée d'une exception.

L’hiérarchie des exceptions



L’hiérarchie des exceptions



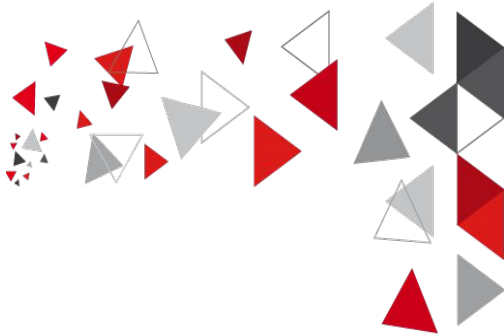
L'arborescence de l'héritage des exceptions commence avec la classe **Throwable**, qui est **la classe de base** de toutes les exceptions en Java.

Les exceptions **Error** indiquent une erreur irrémédiable dans l'environnement d'exécution de Java.(OutOfMemoryError, IOError et VirtualMachineError etc.)

Les exceptions **Exception** sont des exceptions moins graves qui peuvent être gérées par les programmes eux-mêmes.(IOException, SQLException etc.)

RunTimeException est la super-classe de toutes les exceptions normales qui sont des exceptions non vérifiées.

Les exceptions vérifiées et non vérifiées



Les exceptions sont classées en deux catégories principales :

- les exceptions vérifiées (Checked exception)
- les exceptions non vérifiées (Unchecked exception)

Ces catégories jouent un rôle essentiel dans la manière dont nous traitons et gérons les erreurs dans nos programmes.

Les exceptions vérifiées



Les exceptions vérifiées (checked) représentent généralement les événements anticipés qu'une application doit pouvoir gérer. Ils doivent être gérées **explicitement dans le code**, c'est-à-dire qu'elles doivent être capturées par un bloc **try-catch** ou déclarées dans la signature de la méthode.

Les exceptions vérifiées : exemple



Le programme tente d'ouvrir le fichier "file.txt" pour le lire. Si le fichier n'existe pas, ou s'il y a des problèmes d'autorisations, une exception **FileNotFoundException** peut être générée.

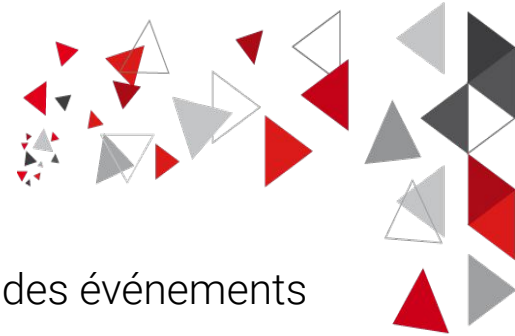
Donc on doit la gérer pour pouvoir compiler notre programme.

```
public class TestException{

   public static void main(String[] args) {
     FileReader fr = new FileReader("file.txt");


   }
}
```

Les exceptions non vérifiées



Les exceptions non vérifiées (unchecked) représentent généralement des événements imprévus auxquels une application ne peut pas faire face.

Ces exceptions sont souvent **évitables** et ne sont donc pas soumises à une vérification de compilation.

Les exceptions non vérifiées : exemple

Dans cet exemple, nous essayons d'accéder à un élément inexistant du tableau **numbers** en utilisant l'indice 3, ce qui provoque une **ArrayIndexOutOfBoundsException**.

Ce code n'affiche aucune erreur au moment de la compilation mais lève une exception lors de l'exécution.

```
public class TestException{

   public static void main(String[] args) {
     int[] numbers = { 1, 2, 3 };
     System.out.println(numbers[3]);
   }
}
```

Output:
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:

Index 3 out of bounds for length 3 at
TestException.main(TestException.java:5)

Gestion des Exceptions



La gestion des exception s'appuie sur le fait que vous savez que la méthode peut avoir un comportement risqué. Si c'est le cas, vous devez écrire explicitement un morceau de code qui prend en compte cette éventualité.

Lorsqu'une méthode possède un comportement risqué, on dit qu'elle peut **générer (lancer ou lever)** une exception.

Gestion des Exceptions



Au cœur de la gestion des exceptions en Java se trouvent cinq concepts clés :

try, catch, finally, throws, et throw. Ces éléments sont essentiels pour gérer les erreurs et les situations exceptionnelles dans vos programmes Java.

On peut gérer les exceptions en Java de **deux manières** :

- Gestion active
- Gestion passive

Gestion active (try-catch)



Le bloc **try-catch** permet de capturer une exception spécifique et de gérer l'erreur de manière appropriée. Si une exception est levée dans le bloc try, le programme passe automatiquement à la section catch où le type d'exception est vérifié et une action est effectuée en conséquence.

Le code potentiellement susceptible de générer une exception est placé à l'intérieur d'un bloc try. Si une exception se produit, l'exécution de ce bloc est interrompue, et le contrôle est transféré au bloc catch pour gérer l’exception.

Gestion active (try-catch) : exemple



La gestion de l'exception n'interrompt pas l'exécution du reste de programme.

```
public class TestException{

   public static void main(String[] args) {
     try{
          System.out.println("Avant incident");
          int[] numbers = { 1, 2, 3 };
          System.out.println(numbers[3]);

     }catch(ArrayIndexOutOfBoundsException ex){
          System.out.println("incident");
     }
          System.out.println("Après incident");
   }
}
```

Output:
Avant incident
incident
Après incident

Gestion active (try-catch) : exemple

On peut déclarer plusieurs blocs **catch,** mais ils doivent être organisés du cas le plus spécifique au cas le plus général.

```
public class TestException{

   public static void main(String[] args) {
     try{
          int[] numbers = { 1, 2, 3 };
          System.out.println(numbers[3]);

     }catch(Exception ex){
          System.err.println(ex.getMessage());
     }catch(ArrayIndexOutOfBoundsException e){ //jamais appelé
          System.err.println(e.getMessage());
     }
   }
}
```

Gestion active (finally)

Le bloc **finally** est utilisé pour contenir du code qui sera exécuté quel que soit le résultat de la tentative (try) et de la gestion d'exception (catch). Seul l'appel de **System.exit()** empêchera l'exécution du bloc finally.

```
public class TestException{

   public static void main(String[] args) {
     try{
          int[] numbers = { 1, 2, 3 };
          System.out.println(numbers[3]);

     }catch(ArrayIndexOutOfBoundsException ex){
          System.err.println(ex.getMessage());
     }finally{
          System.out.println("Le bloc finally s'exécute toujours");
     }
   }
}
```

Gestion active (finally)

La clause **finally** peut être utilisée avec ou sans des clauses catch. Il est couramment utilisé pour effectuer des opérations de nettoyage, telles que la fermeture de fichiers ou la libération de ressources.

```
public class TestException{

   public static void main(String[] args) {
     File file = null;
     try{
          file = new File("temp.txt");
          System.out.println(file.getName());
     }finally{
          if (file != null) {
               file.delete();
          }
     }
   }
}
```

Gestion passive (throws)



Le mot-clé **throws** est utilisé dans la déclaration d'une méthode pour indiquer que la méthode peut lancer certaines exceptions.

Les méthodes qui appellent cette méthode doivent soit capturer ces exceptions avec un bloc try-catch ou déclarer à leur tour qu'elles peuvent lancer ces exceptions avec le mot-clé throws.

Gestion passive (throws) : exemple

```
public class TestException {

     public int divide(int x, int y) throws ArithmeticException {
          return x/y;
     }

     public static void main(String[] args) {
          Util u = new Util();
          try{
               int x = u.divide(10,0);
          }catch(ArithmeticException ex){
               System.err.println(ex.getMessage());
          }
     }
}
```

Lancer une exception (throw)

Vous pouvez lancer manuellement une exception en utilisant le mot-clé **throw** suivi de l'instance de l'exception que vous souhaitez lever.

```
public class TestException {

	public static void main(String[] args) {
		try{
			throw new Exception("Une erreur s'est produite !");
		}catch(ArithmeticException ex){
			System.out.println("Erreur: " + e.getMessage());
		}
	}

	public void tester(int x, int y) throws OhNoException{
		throw new OhNoException("Oh no !");
	}
}
```

Les exceptions personnalisées

il est possible de créer des classes d'exceptions personnalisées en étendant les classes de base **Throwable, Error, ou Exception**. Ces classes d'exceptions personnalisées peuvent être utiles pour signaler des erreurs spécifiques à une application ou à une bibliothèque particulière.

```
public class OhNoException extends Exception{

      public OhNoException(){
      }

      public OhNoException(String messsage){
            super(message);
      }
}
```

Les méthodes de la classe Throwable



String getMessage()	retourne le message d'erreur associé à l'instance de Throwable
void printStackTrace()	Imprime le contenu de la pile dans le fichier d'erreur
StackTraceElement[] getStackTrace()	Retourne un tableau représentant l'état de la pile, au moment où a été levée l'exception. Un élément de stack, contient les informations suivantes : -le nom de la méthode -le numéro de ligne où a été levée l'exception -le nom de la classe -le nom du fichier

Merci pour votre attention

